

Serverraum Überwachungssystem

- **Was wurde entwickelt und warum?**

Wir haben ein Überwachungssystem entwickelt um unseren Serverraum rund um die Uhr sicher und ihm eine geeignete Umgebung zu gewährleisten. Darunter zählen ein Temperatur Messer, einen Abstandsmesser und einen Feuchtigkeitsmesser.

- **Ziele und Anforderungen**

Unser Überwachungssystem soll dafür sorgen, dass der Serverraum nie zu warm oder zu feucht wird. Außerdem sollen bei Arbeiten im Serverraum der Lüfter für die Kühlung ausgeschaltet werden, um das Unfallrisiko zu verringern.

- **Verwendete Technologien**

- Hardware: Arduino, Bread-Board, Ultraschallsensor, LCD-Bildschirm, 12V Lüfter, Kabel, USB-A auf USB-B Kabel
- Software: fritzing

- **Teammitglieder**

- Um die Software hat Lars sich gekümmert und das Skript geschrieben
- Hardwareinstallation hat Luca übernommen
- Dokumentation haben beide zusammen bearbeitet

Inhaltsverzeichnis

1. Technische Dokumentation

1.1 Schaltplan

1.2 Hardwareliste

2. Code-Dokumentation

2.1 Logik-Visualisierung

2.2 Quellcode

3. Testphase und Fehlerbehebung

3.1 Fehlerprotokoll

3.2 Testreihen

4. Benutzeranleitung

4.1 Wie wird das System gestartet?

4.2 Welche Bedienelemente (Taster, Potentiometer) gibt es?

4.3 Wie interpretiert man die Ausgaben auf dem Display

5. Zusammenfassung und Reflektion

5.1 **Ergebnisse:** Was funktioniert gut?

5.2 **Probleme:** Wo gibt es noch offene Baustellen?

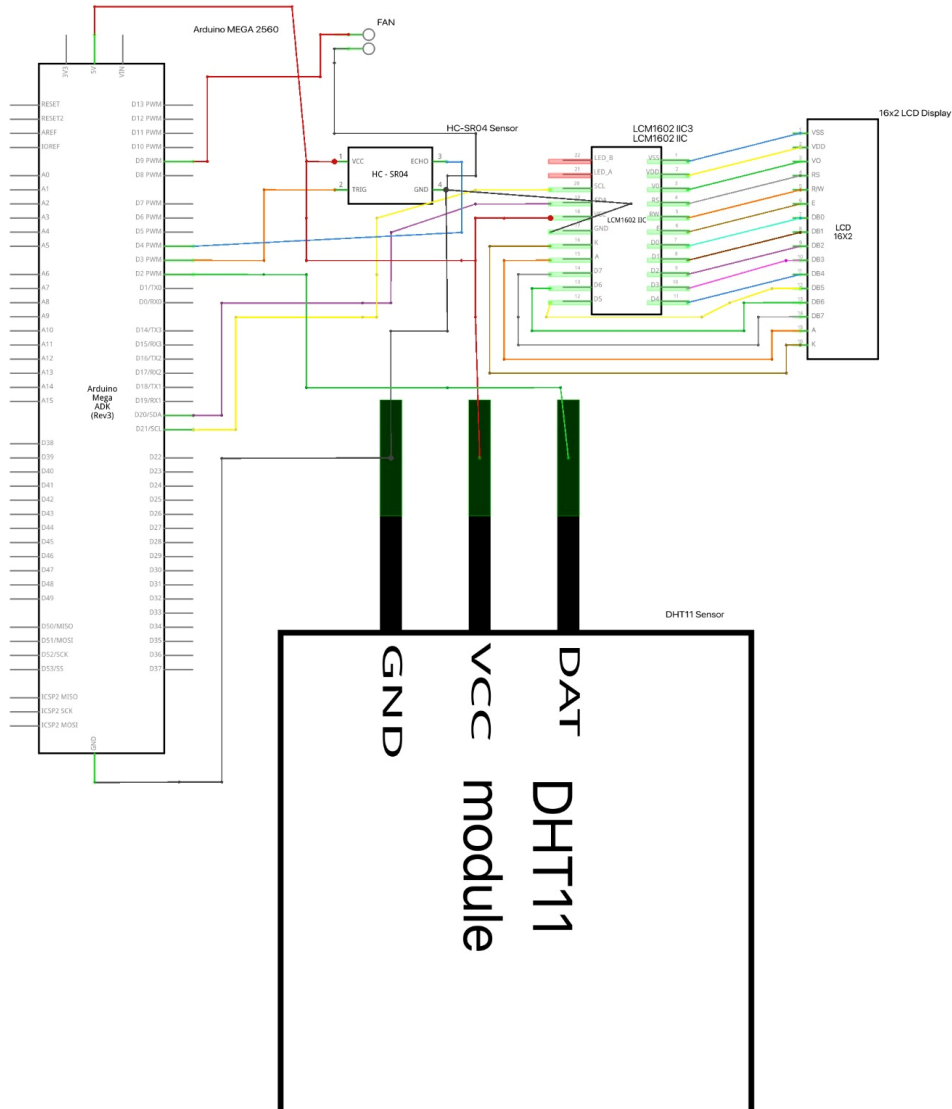
5.3 **Ausblick:** Welche Funktionen könnten in einer Version 2.0 ergänzt werden

6. Anhang

6.1 Anhang zum Quellcode

1. Technische Dokumentation

1.1 Der Schaltplan



1.2 Hardwareliste

1x Arduino Mega 2560

1x TMP36 oder 1x DHT11

1x HC-SR04 Ultraschallsensor

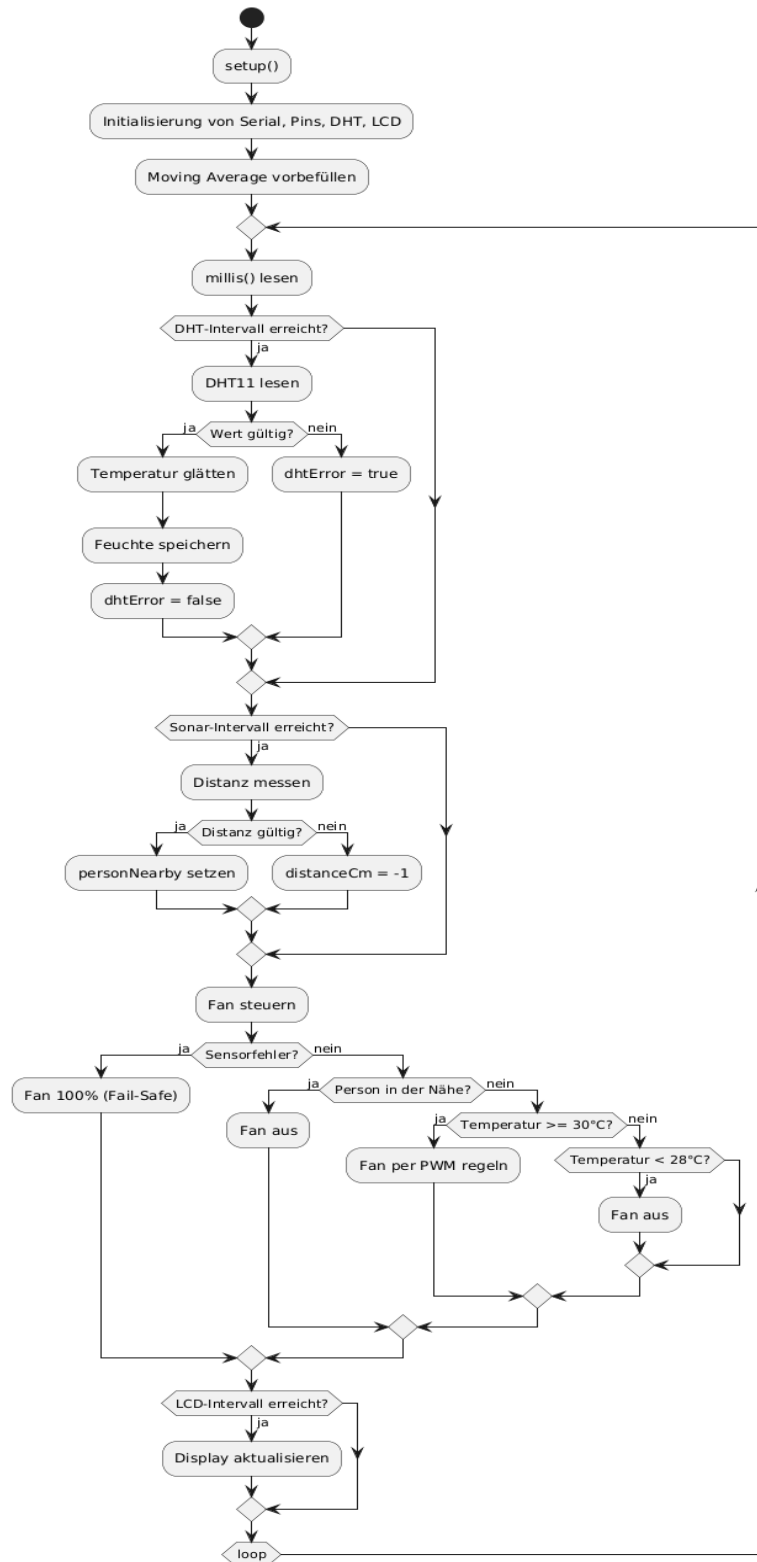
1x I2C LCD 16x2

1x 5V PWM-Lüfter

Verbindungskabel, ggf. Breadboard

2. Code-Dokumentation

2.1 Logik-Visualisierung



2. Code-Dokumentation

2.2 Der Quellcode

- **Temperaturgrenzen:**
 - TEMP_FAN_ON = 30.0°C
 - TEMP_FAN_OFF = 28.0°C (Hysterese)
- **PWM-Mapping:** linear von TEMP_MIN_MAP = 20.0°C bis TEMP_MAX_MAP = 40.0°C auf den Bereich FAN_PWM_MIN bis 255.
- **Personenschutz:** Wenn Distanz < PERSON_DIST_CM (50 cm), wird der Lüfter abgeschaltet.
- **Failsafe:** Bei Sensorausfall läuft Lüfter mit 100%, um Überhitzung zu vermeiden.
- DHT11 darf nur langsam gelesen werden, deshalb INTERVAL_DHT = 2000 ms.
- Gleitender Mittelwert mit 5 Samples (passend zur langsameren DHT11-Rate).
- Zusätzlicher Fehlerzustand dhtError; Failsafe bei DHT- **oder** Sonarfehler.

Wichtige Funktionen

- smoothedTemperature() – Mittelwertbildung.
- updateFan() – Failsafe bei dhtError || distanceCm == -1.
- updateDisplay() – zeigt Temperatur, Feuchte, Lüfter und Distanz; im Fehlerfall „ERR“.

3. Testphase und Fehlerbehebung

3.1 Fehlerprotokoll

1. LCD bleibt leer

- Ursache: falsche I2C-Adresse.
- Lösung: 0x27 auf 0x3F ändern.

2. DHT11 liefert NaN

- Ursache: zu schnelle Abfrage / Leitungsproblem.
- Lösung: 2-Sekunden-Intervall einhalten, Verkabelung prüfen.

3. HC-SR04 keine Werte

- Ursache: Echo-Timeout oder Reichweite überschritten.
- Lösung: Timeout/Verkabelung prüfen; System setzt automatisch Failsafe.

4. Lüfter startet nicht zuverlässig

- Ursache: PWM-Startwert zu niedrig.
- Lösung: FAN_PWM_MIN erhöhen.

3.2 Testreihen

- **Schwellwerttest:** Sensor erwärmen (z. B. Föhn), prüfen, ob Lüfter ab ~30°C startet.
- **Hysteresis-Test:** Temperatur langsam senken, prüfen, ob Lüfter erst unter 28°C ausgeht.
- **Personenschutz-Test:** Hand vor HC-SR04 (<50 cm), Lüfter muss stoppen.
- **Failsafe-Test Sonar:** Echo-Leitung lösen/simulieren, Lüfter muss 100% laufen.
- **Failsafe-Test DHT11:** DHT11 trennen, Lüfter muss 100% laufen.
- **Langzeittest:** 30–60 Minuten Betrieb, stabile Anzeige und keine Hänger.

4. Benutzeranleitung

4.1 Wie wird das System gestartet?

1. Hardware gemäß Schaltplan anschließen.
2. Passenden Sketch hochladen:
 - serverroom_fan_control-TMP36.ino oder
 - serverroom_fan_control_dht11.ino
3. Arduino mit Strom versorgen.
4. Nach Initialisierung erscheinen Messwerte auf dem LCD.

4.2 Welche Bedienelemente (Taster, Potenziometer) gibt es?

- Es gibt keine aktiven Bedienelemente; das System arbeitet automatisch.
- Optional können Grenzwerte im Code angepasst werden:
 - TEMP_FAN_ON, TEMP_FAN_OFF
 - PERSON_DIST_CM
 - FAN_PWM_MIN

4.3 Wie interpretiert man die Ausgabe auf dem Display?

- Zeile 1: Temperatur (und bei DHT11 auch Luftfeuchte)
- Zeile 2: Lüfterleistung in % und Distanz
- Sonderanzeigen:
 - PERSON DETECTED! → Lüfter aus
 - SENSOR ERROR! bzw. ERR-Werte → Failsafe aktiv

5. Zusammenfassung und Reflektion

5.1 Ergebnisse: Was funktioniert gut?

- Das System erfüllt die Kernfunktion: temperaturabhängige Lüftersteuerung.
- Sicherheits- und Failsafe-Mechanismen erhöhen die Betriebssicherheit.
- Die modulare Struktur (Lesen, Regeln, Anzeigen) verbessert Wartbarkeit und Erweiterbarkeit.

5.2 Probleme: Wo gibt es noch offene Baustellen?

- Ohne Kalibrierung können absolute Temperaturwerte je nach Sensor leicht abweichen.
- DHT11 ist relativ ungenau und langsam.
- HC-SR04 kann in engen/geometrisch ungünstigen Umgebungen Fehlmessungen erzeugen.

5.3 Ausblick: Welche Funktionen könnten in einer Version 2.0 ergänzt werden?

- Austausch DHT11 gegen DHT22 oder BME280 für genauere Messungen.
- Logging auf SD-Karte (Temperaturverlauf, Fehlerereignisse).
- Alarmfunktion (Buzzer/LED/Nachricht) bei Grenzwertüberschreitung.
- Web-Dashboard (z. B. per ESP32) für Fernüberwachung.
- Manuelle Betriebsmodi (Auto/Manuell/Service).

6. Anhang

6.1 Der Verwendete Quellcode: [gvid-serverraum-ueberwachung/serverroom_fan_control_dht11.ino](#) at main · lorzfr/gvid-serverraum-ueberwachung